

Codifica delle informazioni e Macchine Elementari (Richiami)

Reti Logiche

Dipartimento di Informatica e Sistemistica
Università degli Studi di Napoli “Federico II”



Tassonomia dei circuiti digitali

- I circuiti digitali possono essere classificati in due categorie
- **Circuiti combinatori**
 - » Il valore delle uscite ad un determinato istante dipende unicamente dal valore degli ingressi in quello stesso istante.
- **Circuiti sequenziali**
 - » Il valore delle uscite in un determinato istante dipende sia dal valore degli ingressi in quell'istante sia dal valore degli ingressi in istanti precedenti (concetto di tempo)
 - » Per definire il comportamento di un circuito sequenziale è necessario tenere conto della storia passata degli ingressi del circuito (concetto di stato)



Macchine combinatorie (1/4)

Reti logiche con n ingressi $x_1, x_2, \dots, x_n$ e m uscite $y_1, y_2, \dots, y_m$ che realizzano la corrispondenza:

$$y_1 = f_1(x_1, x_2, \dots, x_n)$$

.....

$$y_m = f_m(x_1, x_2, \dots, x_n)$$



Macchine combinatorie (2/4)

- Una macchina combinatoria è una rete logica con n ingressi ($x_1, x_2, \dots, x_n$) ed m uscite ($y_1, y_2, \dots, y_m$) ed è tale che ad ogni insieme di valori degli ingressi corrisponde un preciso insieme di valori delle uscite
- Il comportamento di una rete combinatoria $n \times m$ può essere descritto tramite:
 - » una tabella di verità in cui viene specificato il valore dell'uscita per ognuna delle possibili combinazioni dei valori degli ingressi
 - » m funzioni booleane, una per ogni uscita, ciascuna delle quali esprime il valore della corrispondente variabile di uscita in funzione delle n variabili di ingresso



Macchina combinatoria (3/4)

- In una macchina combinatoria i valori delle uscite dipendono esclusivamente dai valori degli ingressi
 - » macchina combinatoria ideale: tale dipendenza è istantanea
 - » macchina combinatoria reale: presenza di ritardo tra l'istante in cui c'è una variazione in uno degli ingressi e l'istante in cui l'effetto di questa variazione si manifesta sulle uscite
- E' importante notare come
 - » ciascuna y_i può essere decomposta in funzioni componenti
 - » due distinte y_i possono contenere una identica funzione componente
- Ciò comporta, ad esempio, una potenziale diminuzione di porte elementari rispetto ad una realizzazione indipendente delle y_i



Macchina combinatoria 4/4

- Mentre ad ogni rete combinatoria corrisponde un'unica tabella di verità, ad una tabella di verità possono corrispondere più reti combinatorie
- Nel procedimento di sintesi ci possiamo quindi porre particolari obiettivi come ad esempio:
 - » utilizzare esclusivamente circuiti elementari di un certo tipo
 - » progettare un circuito che abbia il minimo numero di porte logiche



Codifica



Codifica delle informazioni: sul piano formale

- *Rappresentazione* di informazioni appartenenti a un insieme finito D
- Funzione *iniettiva* dall'insieme D (*dominio*) a un insieme direttamente manipolabile R (*codominio*)

$$c : D \rightarrow R$$

$$|R| \geq |D|$$

- La funzione c è detta *codifica* o *rappresentazione* delle informazioni appartenenti a D



Definizioni

- Alfabeto origine T : il dato da rappresentare appartiene ad un tipo $T = (x_1, \dots, x_n)$
- Alfabeto in codice E : il dato è rappresentato mediante dati di tipo $E = (a_1, \dots, a_k)$
- Tabella Codice C : la codifica di T mediante E è un'applicazione C , detta *tabella codice*, che trasforma ciascun elemento $x_i \in T$ in una stringa di lunghezza l_i di elementi $a_i \in E$, detta *parola codice*



Codifica a lunghezza fissa

Se si pone $l_i = m = \text{costante}$ per tutti gli elementi di T , si ottiene una codifica a lunghezza fissa. In tal caso il codice è fissato facendo corrispondere a ciascun elemento $x_i \in T$ una delle k^m disposizioni con ripetizione dei k simboli di tipo E sugli m posti della stringa e quindi:

$$K^m \geq N$$

da cui

$$m = \lceil \log_k N \rceil$$

Per codificare un dato di cardinalità N mediante un alfabeto di k simboli è necessaria una stringa di lunghezza minima m

$$l \geq m$$

l lunghezza del codice



Codifica a lunghezza fissa

CODICE INCOMPLETO : se $l = m$ ma N non è potenza di k , il codice viene detto incompleto e la differenza $k^m - N$ fornisce il numero di parole codice non assegnate, cioè non associate ad alcun elemento dell'alfabeto origine. (vedi es. 1, 2)

CODICE RIDONDANTE : si ottiene per $l > m$ adoperando più caratteri dell'alfabeto in codice di quanto strettamente necessari. Questi codici vengono utilizzati per rilevare ed eventualmente correggere errori dovuti ad alterazioni del dato.

CODICE RIDONDANTE → CODICE INCOMPLETO



Codifica a lunghezza variabile

La lunghezza l_i del codice è funzione di ciascun elemento x_i :

$$l_i = f(x_i)$$

Proprietà fondamentale è quella che la (o le) parola più corta è individuata da un particolare codice che non si ritrova come sequenza iniziale in quelle più lunghe: è la lunghezza stessa che deve essere riconosciuta nel contesto della parola-codice.

L'uso di questo tipo di codifica è giustificato quando gli elementi del tipo T (alfabeto origine) non hanno tutti la stessa probabilità di occorrenza.



Codifica a lunghezza variabile

Dato l'insieme

$$T = (x_1, \dots, x_n)$$

Dette

$$p_1, \dots, p_n$$

le probabilità di occorrenza (frequenza) dei rispettivi elementi di T

la lunghezza l_i viene scelta in modo da minimizzare la lunghezza media L_m del codice

$$L_m = \sum_{i=1,n} (p_i * l_i)$$

Effettuata la ricerca della n-pla di valori l_i che rende minima L_m si ottiene un codice a lunghezza variabile a minima ridondanza. Le medesime informazioni codificate con un codice a lunghezza fissa avrebbero richiesto un codice di lunghezza maggiore di L_m



Codifica a lunghezza variabile

- Data la rappresentazione $c : D \rightarrow R$, l'insieme R può essere costituito da stringhe di lunghezza differente
- Esempio (stringhe di cifre da 0 a 3):

<i>Informazione</i>	<i>codice</i>	<i>Informazione</i>	<i>codice</i>	<i>Informazione</i>	<i>codice</i>
Casa	0	Banca	32	Andrea (C)	3310
Genitori	1	Paolo	3300	Andreani	3311
Segretaria	2	Anna	3301	Marocco	3312
Direttore	30	Mario	3302	Daita	3313
Taxi	31	Mario (C)	3303	Luchini	3320



Codifica a lunghezza variabile

- La corrispondenza viene decisa tenendo conto della frequenza con cui vengono usati i valori in D
- Vantaggi:
 - » Risparmio di spazio nella memorizzazione
 - » Risparmio di tempo nella trasmissione



Il processo di codifica

- Esempi di codici intermedi completi
 - » *Ottale*
 - » *Esadecimale*
- Esempio di codice intermedio incompleto
 - » *8-4-2-1 (BCD)*

Codice ottale

cifra	C ₂	C ₁	C ₀
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

NON UNIVOCAMENTE
DEFINITO

cifra	8-4-2-1
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Codice esadecimale

cifra	C ₃	C ₂	C ₁	C ₀	cifra	C ₃	C ₂	C ₁	C ₀
0	0	0	0	0	8	1	0	0	0
1	0	0	0	1	9	1	0	0	1
2	0	0	1	0	A	1	0	1	0
3	0	0	1	1	B	1	0	1	1
4	0	1	0	0	C	1	1	0	0
5	0	1	0	1	D	1	1	0	1
6	0	1	1	0	E	1	1	1	0
7	0	1	1	1	F	1	1	1	1



Codici Ridondanti

Un codice a lunghezza fissa l per la codifica di un dato di cardinalità N con un alfabeto codice di cardinalità k si dice ridondante se:

$$K^l > N$$

$$(l > m, \text{ con } m : K^m = N)$$

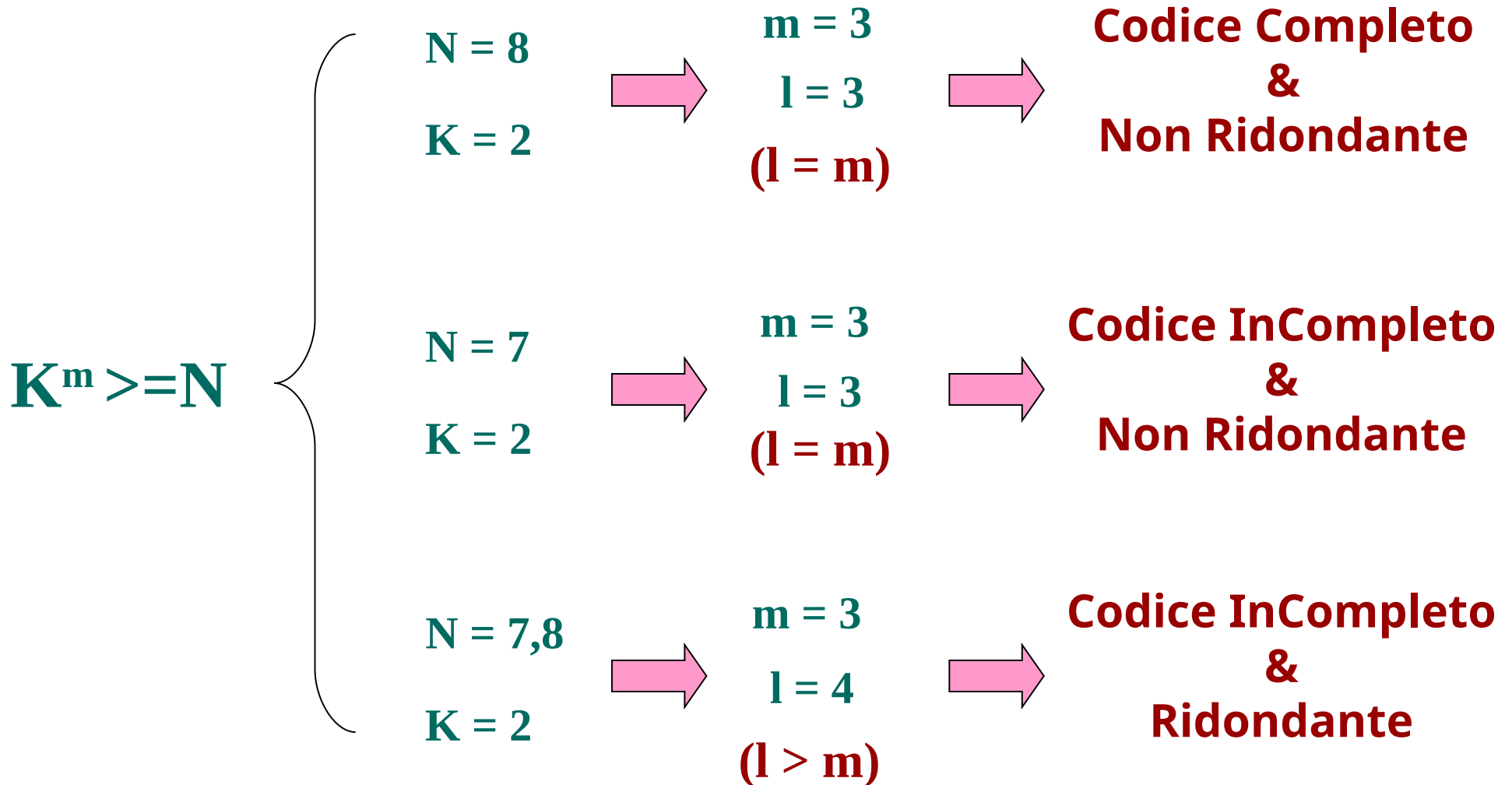
Una misura della ridondanza è:

$$r = 1 - [(\log_k N)/l] \in [0,1]$$

r è uguale a 0 per $\log_k N = l$ mentre tende ad 1 per l che tende ad infinito



Codici Ridondanti



Rappresentazione di Codici mediante tabelle

$T = (x_1, x_2, \dots, x_n)$ alfabeto origine

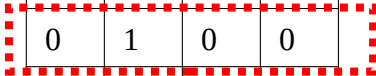
$E = (a_1, a_2, \dots, a_k)$ alfabeto destinazione

Es. Codice BCD (Binary Coded Decimal)

	3	2	1	0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

Dato

Parola Codice



Rappresentazione Decodificata

- L'insieme ha cardinalità N
- Le parole codice hanno lunghezza $m = N$
- Ad ogni parola codice è associato un solo bit 1
- Motivazioni



Esempio di rappresentazione Decodificata

Dato	Codice BDC	Codice Decodificato
0	0000	1000000000
1	0001	0100000000
2	0010	0010000000
3	0011	0001000000
...	...	
...		
9	1001	0000000001

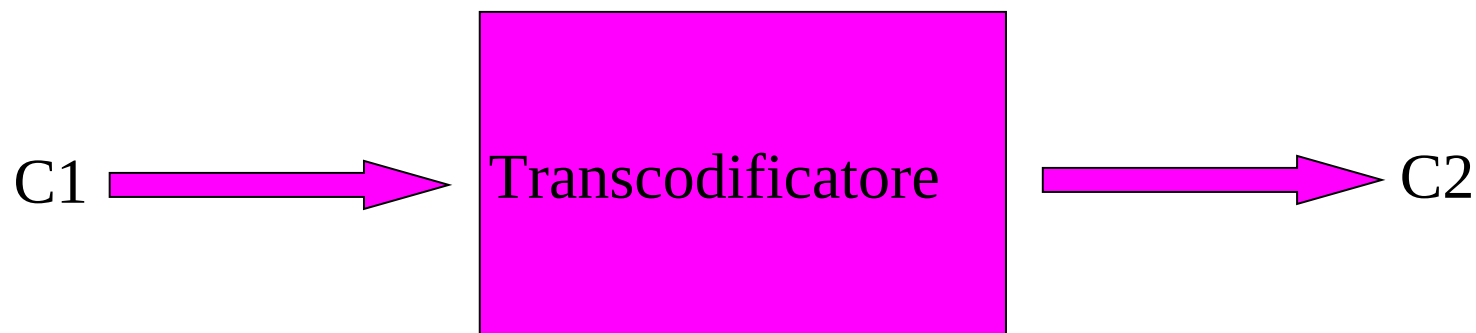


Transcodificatore



Transcodificatore

- In alcune circostanze si può verificare che lo stesso dato sia rappresentato mediante codici diversi.
- Detti C1 e C2 due codici definiti sullo stesso insieme (cioè costituito dallo stesso numero di parole codice), si dice TRANSCODIFICATORE una macchina che consente di associare a ciascuna parola del codice C1 la corrispondente parola del codice C2.
 - » Transcodificatore per visualizzatore a 7 segmenti.



Trascodificatore per visualizzatore a 7 segmenti

- Uno degli indicatori visivi più comuni è l'**indicatore a 7 segmenti**.
- Ogni simbolo è formato da sette segmenti ognuno dei quali è un Led che può essere acceso da un segnale digitale.
- Un **BCD-To-Seven-Segment-Decoder** riceve in ingresso un simbolo decimale in BCD e genera l'appropriata uscita selezionando i segmenti che devono essere accesi per mostrare su display il simbolo decimale.



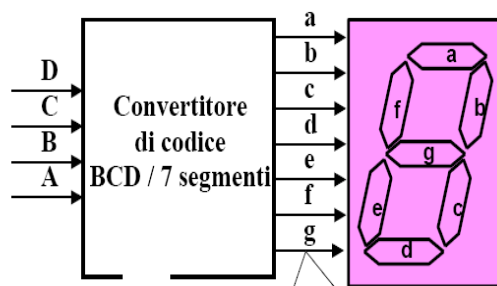
(a) Segment designation



(b) Numeric designation for display

Trascodificatore per visualizzatore a 7 segmenti

- Le 7 uscite le indichiamo con (a,b,c,d,e,f,g) selezionando i corrispondenti segmenti. Si hanno:
 - » 4 input: A B C D.
 - » 7 output: a b c d e f g.
- La tabella di verità sarà pertanto ⇒



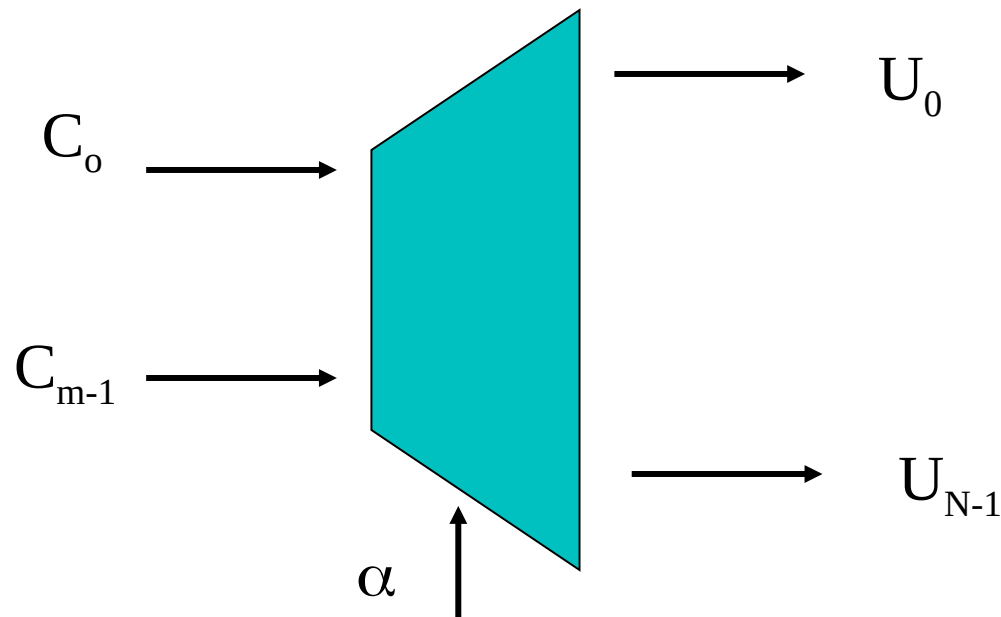
BCD Input				Seven-Segment Decoder						
A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
All other inputs				0	0	0	0	0	0	0

Decoder ed Encoder



Decodificatore (decoder)

- Un decodificatore è una macchina che riceve in ingresso una parola codice (C) e presenta in uscita la sua rappresentazione decodificata (linee $U_0, \dots, U_{N-1}$)

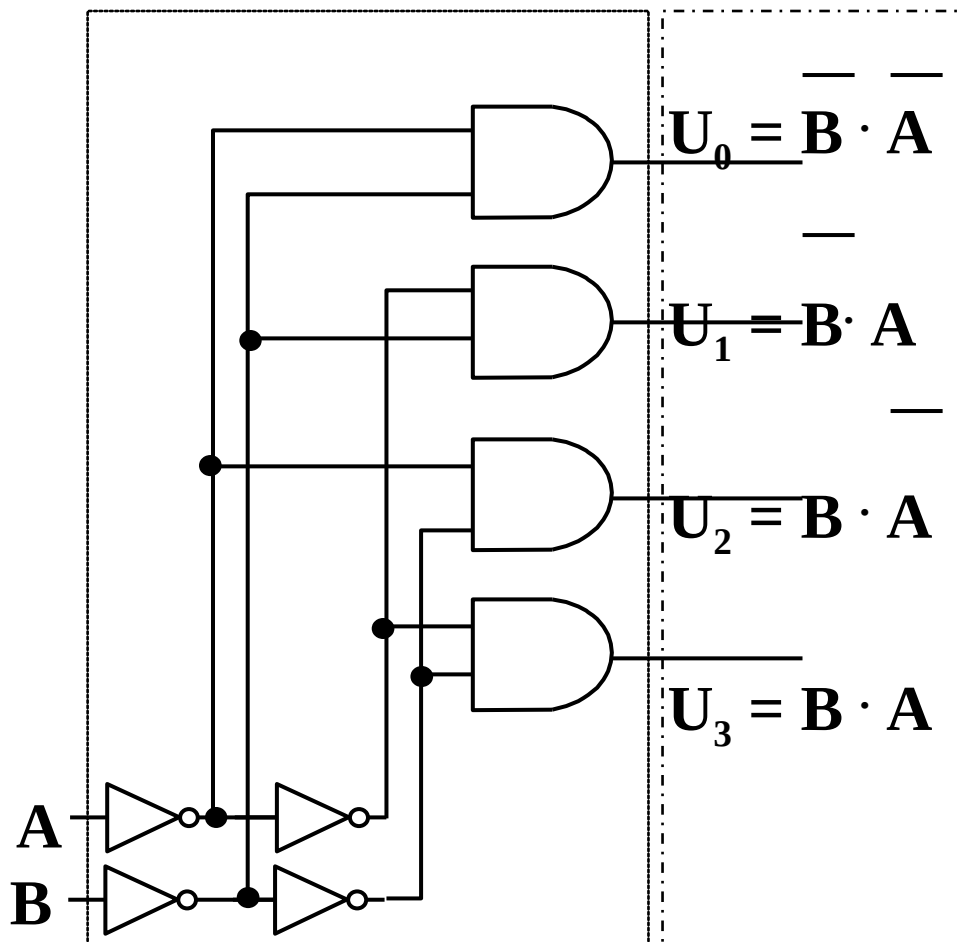


Sintesi della trascodifica da binario a 1 su N

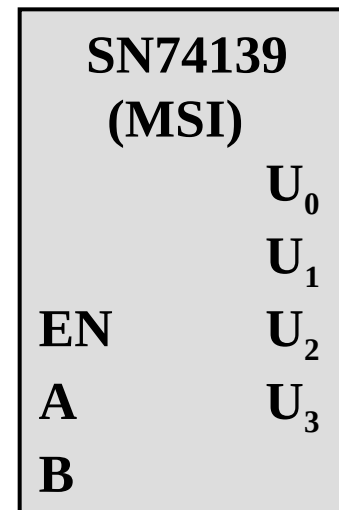
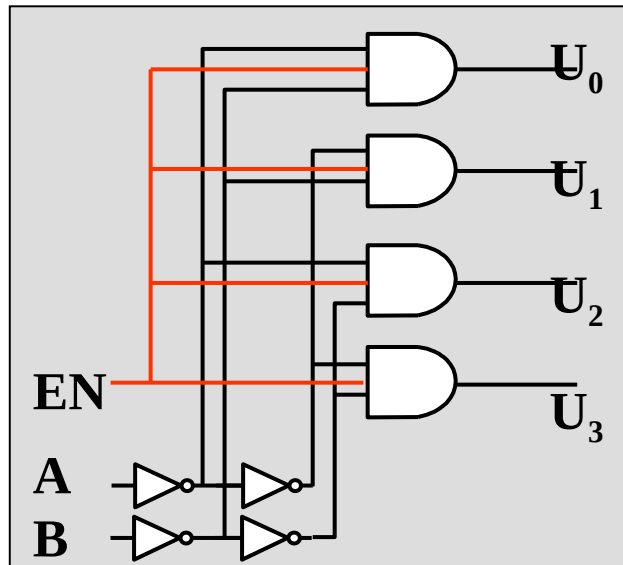
Abbiamo un insieme sorgente di 4 elementi. Sono necessari 2 bit (A,B) per codificarlo. Nella rappresentazione decodificata servono invece 4 bit ($U_0U_1U_2U_3$).

Esempio: Transcodifica 2:4

	B	A	U_0	U_1	U_2	U_3
1						
2	0	0	1	0	0	0
3	0	1	0	1	0	0
4	1	0	0	0	1	0
	1	1	0	0	0	1



Il circuito integrato DECODER

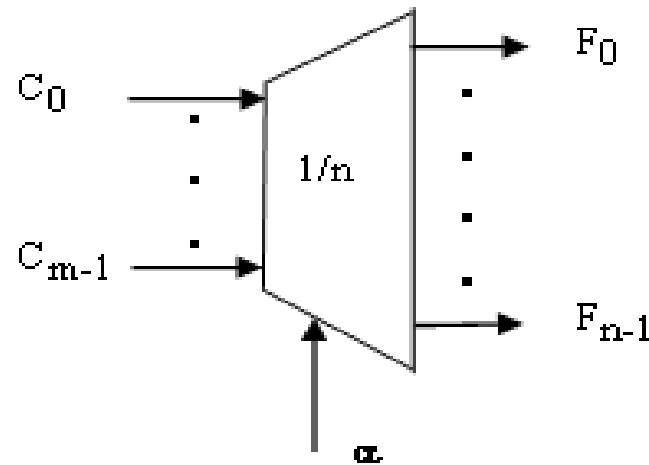


Quando EN=1 (segnale di abilitazione), vale 1 l'uscita il cui pedice, in decimale, corrisponde al numero binario in ingresso (A bit di minor peso).

Decodificatore binario (1/2)

➤ Decoder Binario

- » m ingressi binari su cui è inviata la parola codice,
- » ingresso di abilitazione α (*opzionale*)
- » n uscite ($n \leq 2^m$)



Decodificatore binario (2/2)

➤ Decoder Binario Completo

» $n=2^m$

» $F_0 = P_0; F_1 = P_1; \dots; F_{n-1} = P_{n-1}$

» **Equazione**

$$F_i = a \cdot P_i \quad i \in [0, 2^n-1] \quad (1\text{-att})$$

» è indipendente dal particolare codice da decodificare

➤ Decoder Binario Incompleto

» $n < 2^m$

» Si pone $n'=2^m$ e si realizza la rete come sottorete della rete completa

» è dipendente dal particolare codice da decodificare



Decodificatore incompleto

- Decoder ad hoc per il codice 8-4-2-1 (per cifre decimali)

$$\begin{aligned}
 F_0 &= \overline{C_3} \overline{C_2} \overline{C_1} \overline{C_0}; & F_1 &= \overline{C_3} \overline{C_2} \overline{C_1} C_0; & F_2 &= \overline{C_3} \overline{C_2} C_1 \overline{C_0}; \\
 F_3 &= \overline{C_3} \overline{C_2} C_1 C_0; & F_4 &= \overline{C_3} C_2 \overline{C_1} \overline{C_0}; & F_5 &= \overline{C_3} C_2 \overline{C_1} C_0; \\
 F_6 &= \overline{C_3} C_2 C_1 \overline{C_0}; & F_7 &= \overline{C_3} C_2 C_1 C_0; & F_8 &= C_3 \overline{C_0}; & F_9 &= C_3 C_0
 \end{aligned}$$

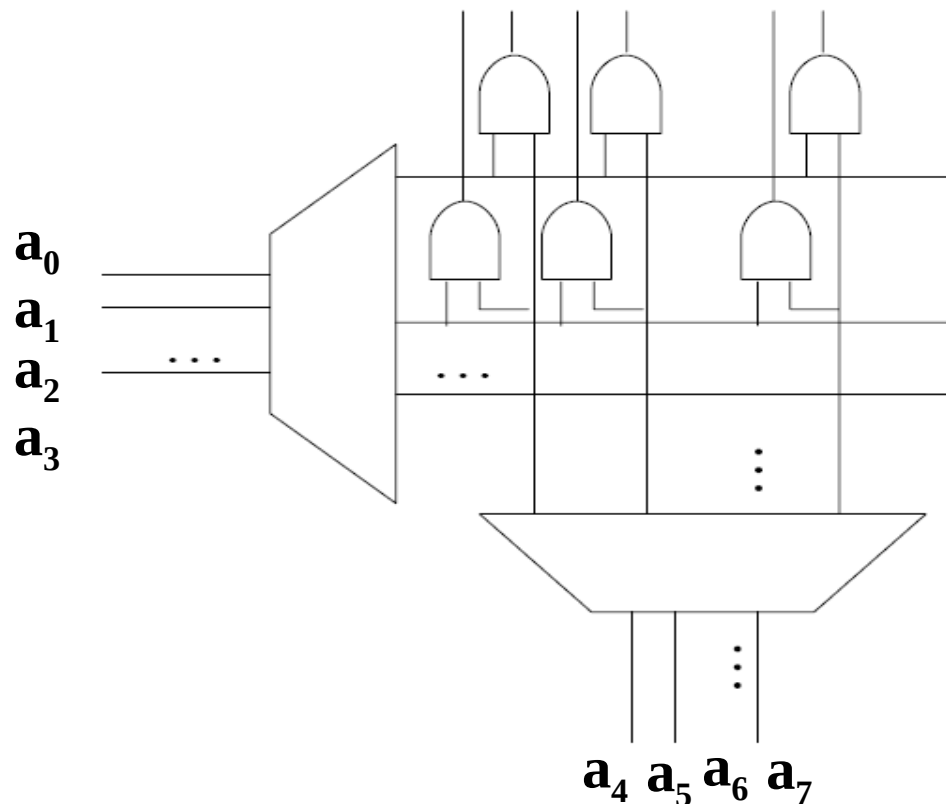
$C_3 C_2$					
$C_1 C_0$		00	01	11	10
		F_0	F_4	-	F_8
01		F_1	F_5	-	F_9
11		F_3	F_7	-	-
10		F_2	F_6	-	-

La rete così ottenuta ha un numero di porte *and* uguale alla rete realizzata come sottorete di rete completa, ma non tutte le sue porte sono a 4 ingressi e quindi è di costo *Ci* inferiore.



Decodificatori composti

- Più decoder possono essere composti...
 - » Semiselezione (4/16)

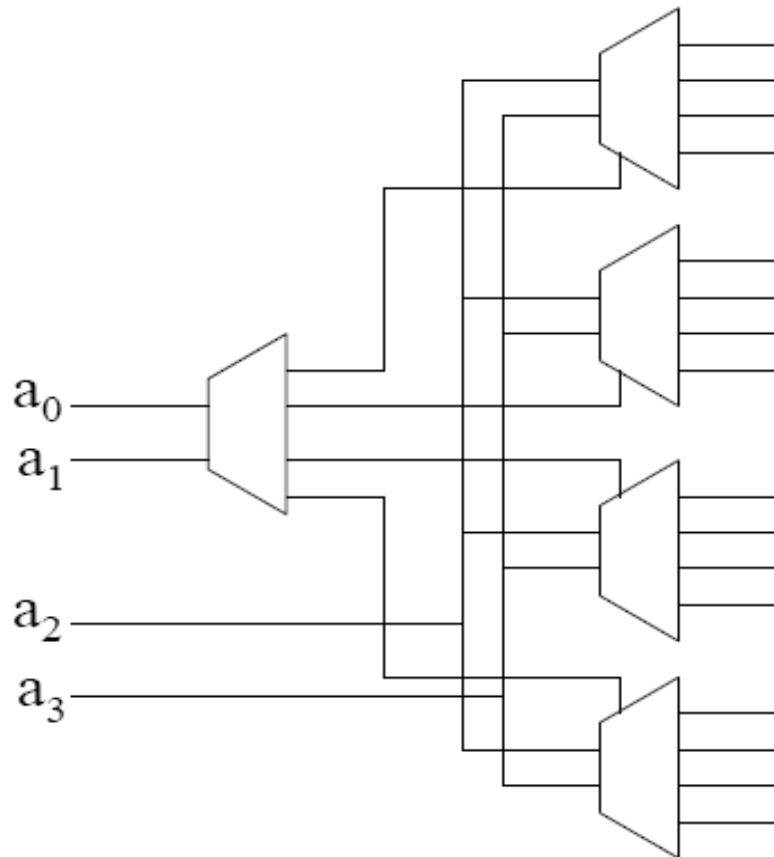


**Es: 2 decodificatori
di metà ampiezza (4
uscite)**

**1 porta AND per
ciascuna delle uscite**

Decodificatori composti

» Schema ad albero



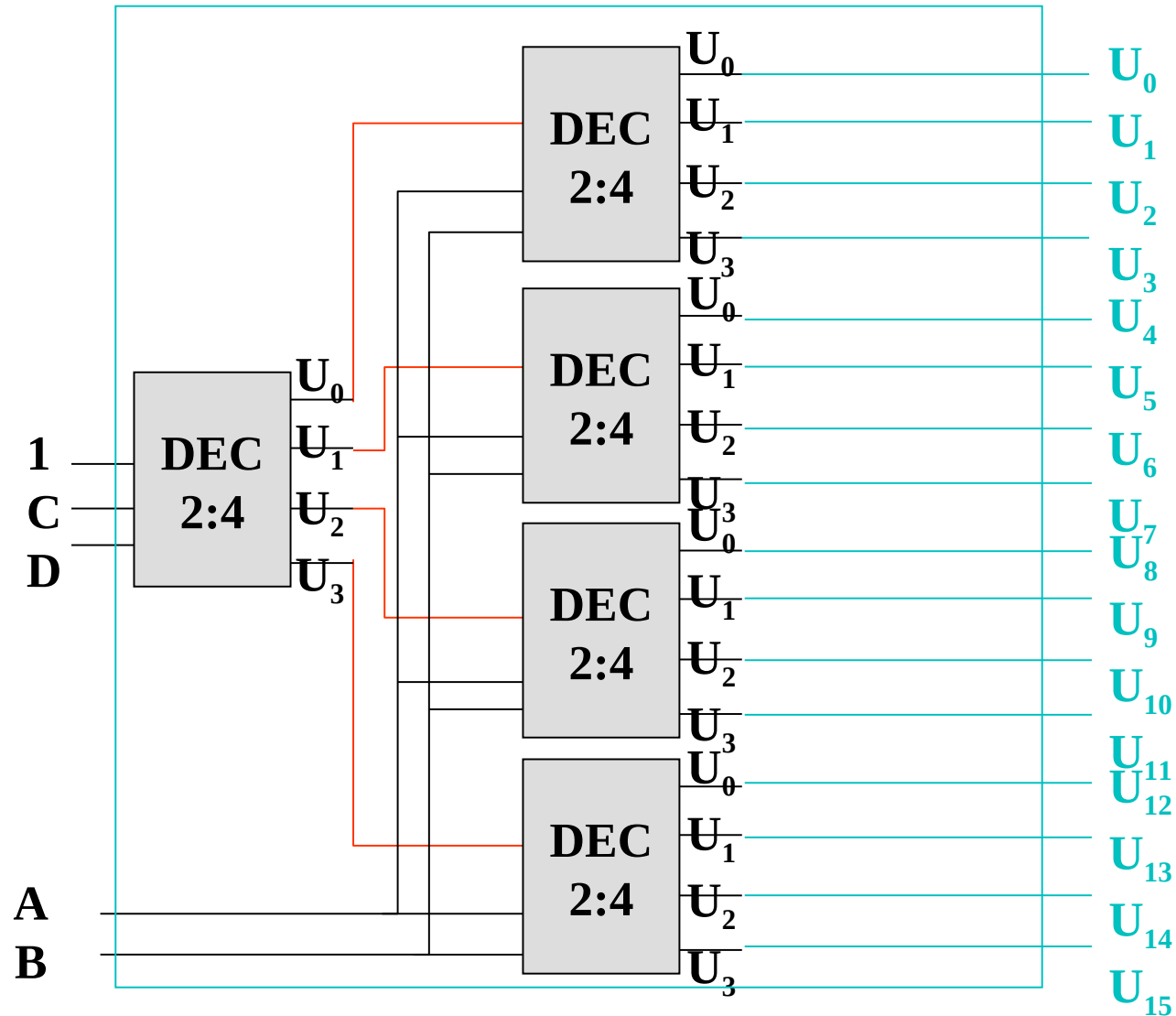
E' possibile realizzare un decoder di

- n uscite (Es. 16)
- $\log_2 n$ ingressi (Es. 4)

attraverso decoder a

- m uscite (Es. 4)
- $\log_2 m$ ingressi (Es. 2)

Composizione modulare di Decoder 4:16



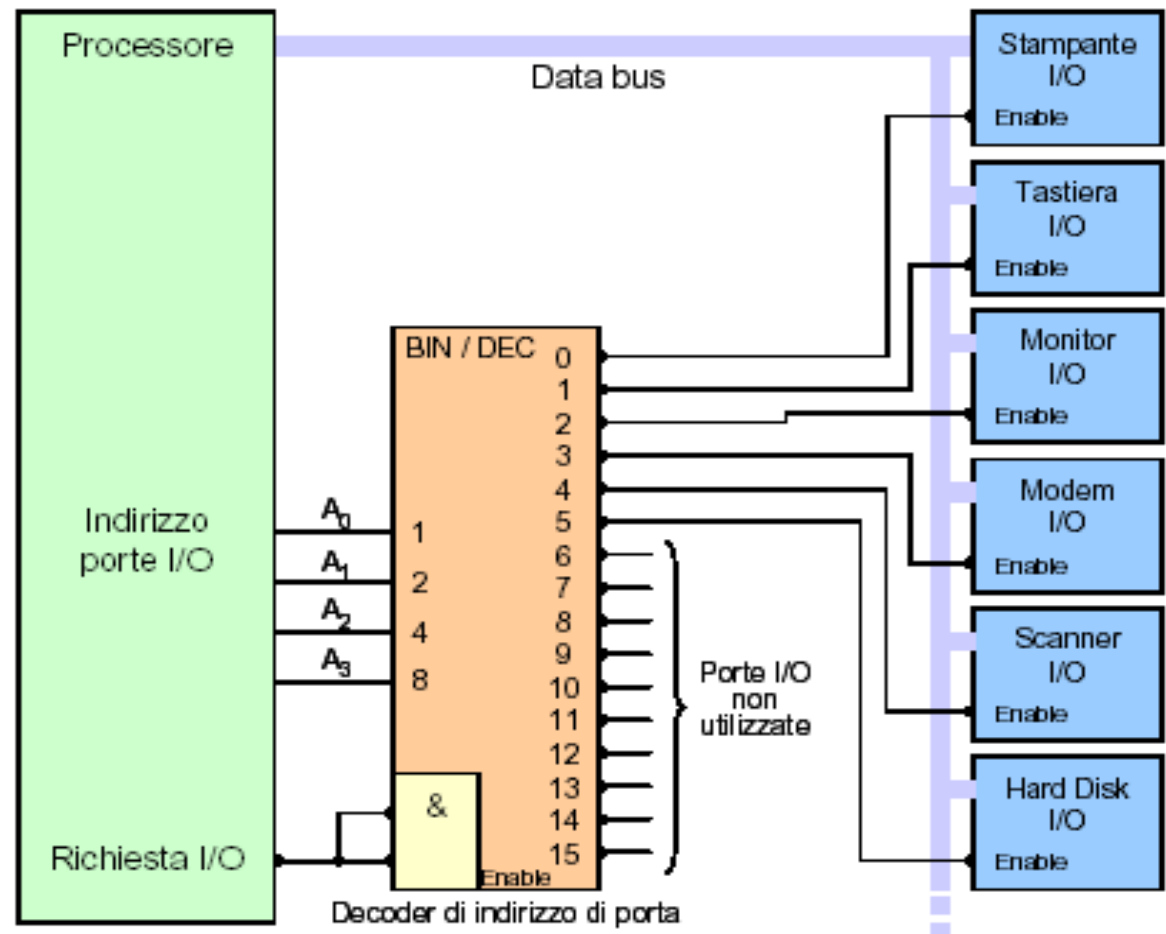
Esempio (semplificato) di utilizzo di Decoder (1/2)

- I decoder sono utilizzati in moltissime applicazioni.
- Un esempio è la selezione dell'input/output da parte del processore.
- Ogni porta di I/O possiede un indirizzo (univoco).
- Quando il processore vuole comunicare con una determinata periferica esso produce il codice dell'indirizzo della porta di I/O al quale la periferica è connessa.
- L'indirizzo (binario) della porta è fornito in input al decoder il quale attraverso il suo output abilita la corrispondente porta di I/O.



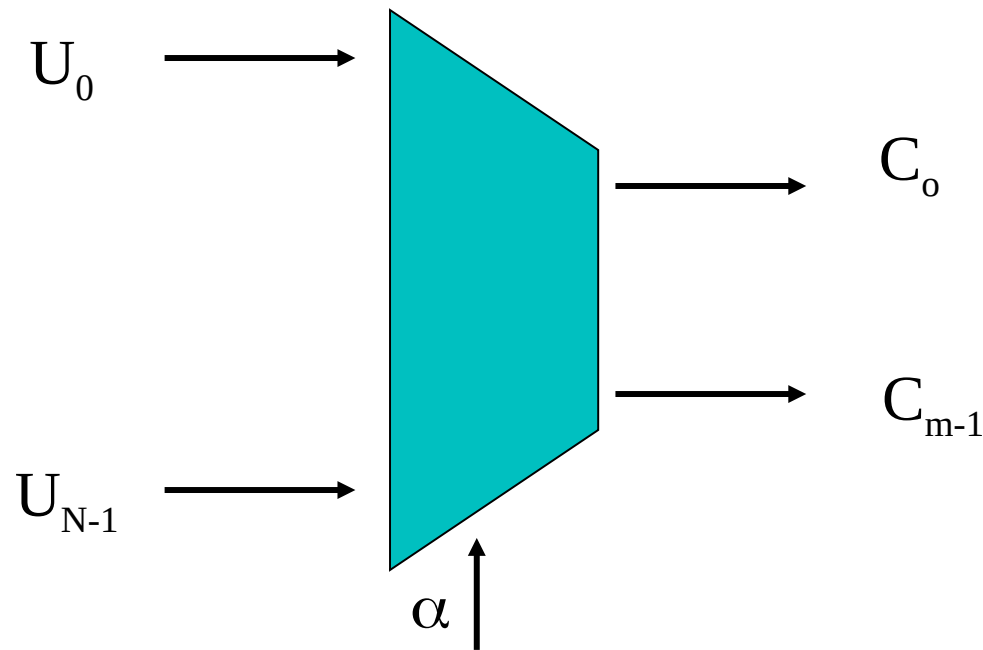
Esempio (semplificato) di utilizzo di Decoder (2/2)

- Come mostrato in Figura l'informazione binaria è trasferita su un **bus di dati**, che è un set di fili paralleli.
- **Esempio:** un bus di 8-bit consiste in 8 fili paralleli che trasferiscono un byte di dati alla volta.
- Il bus è connesso a tutte le porte I/O, ma ogni dato in transito passerà attraverso la porta soltanto se questa è abilitata dal decoder.



Codificatore (Encoder)

- Un codificatore è una macchina che riceve in ingresso una rappresentazione decodificata (linee $U_0, \dots, U_{N-1}$) e fornisce in uscita la parola codice associata a U_i se $U_i=1$ (più in generale se è attivo).

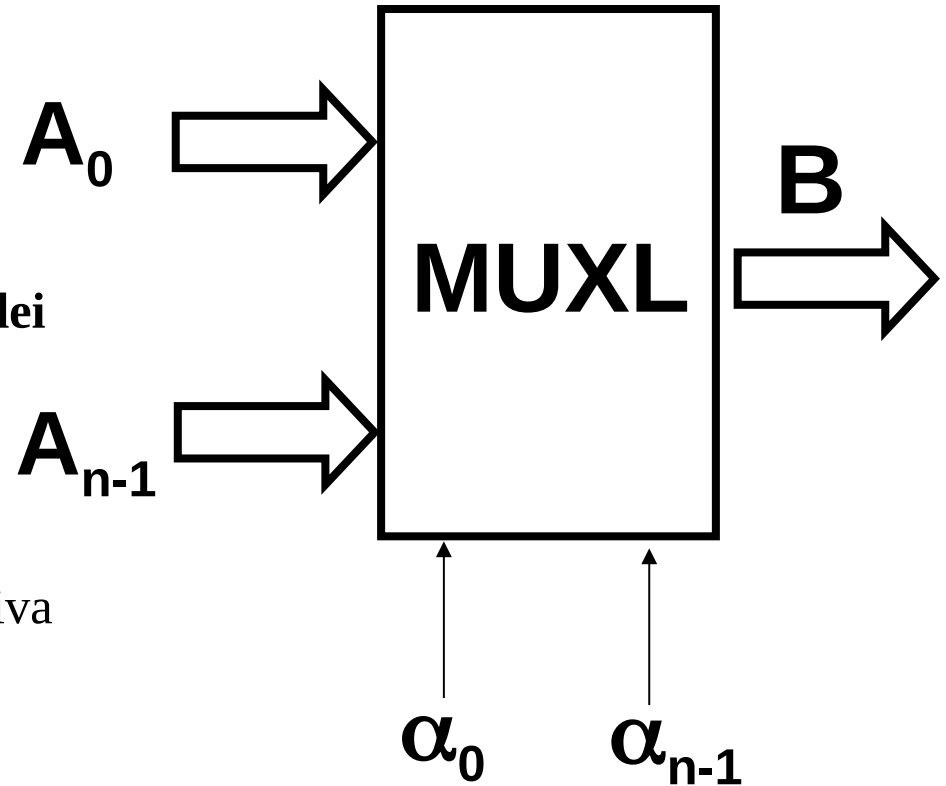


Multiplexer e Demultiplexer



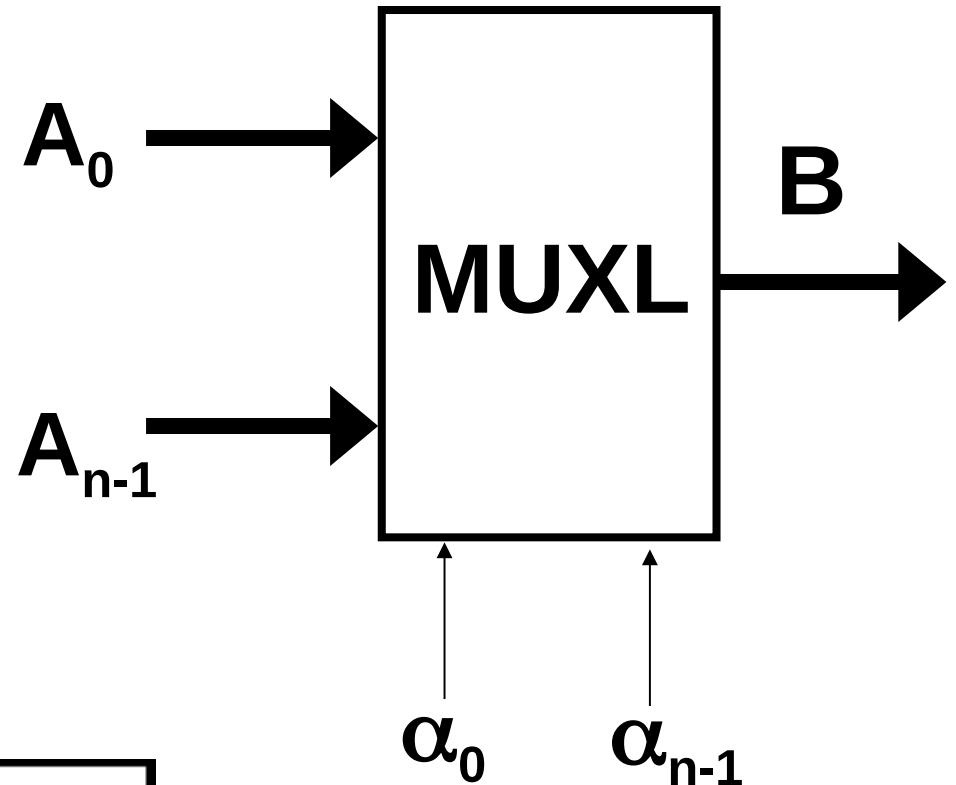
Multiplexer lineare

- Un *Multiplexer lineare* (ML) è una macchina con:
 - » n ingressi-dati ($A_0, \dots, A_{n-1}$)
 - » n segnali binari di selezione ($\alpha_0, \dots, \alpha_{n-1}$), **dei quali al più uno è attivo**
 - » una uscita-dati B , che assume
 - ◆ valore A_i se è attivo α_i
 - ◆ neutro se nessuna delle selezioni è attiva
- E' una macchina che viene utilizzata quando più linee devono essere convogliate verso un'unica linea di uscita (bus)



Multiplexer binario

- Se i dati A_i e B sono quelli che viaggiano su un bus ...
 - » si parla di multiplexer o demultiplexer
- Se i dati A_i e B sono semplici bit ...
 - » si parla di multiplexer o demultiplexer binario

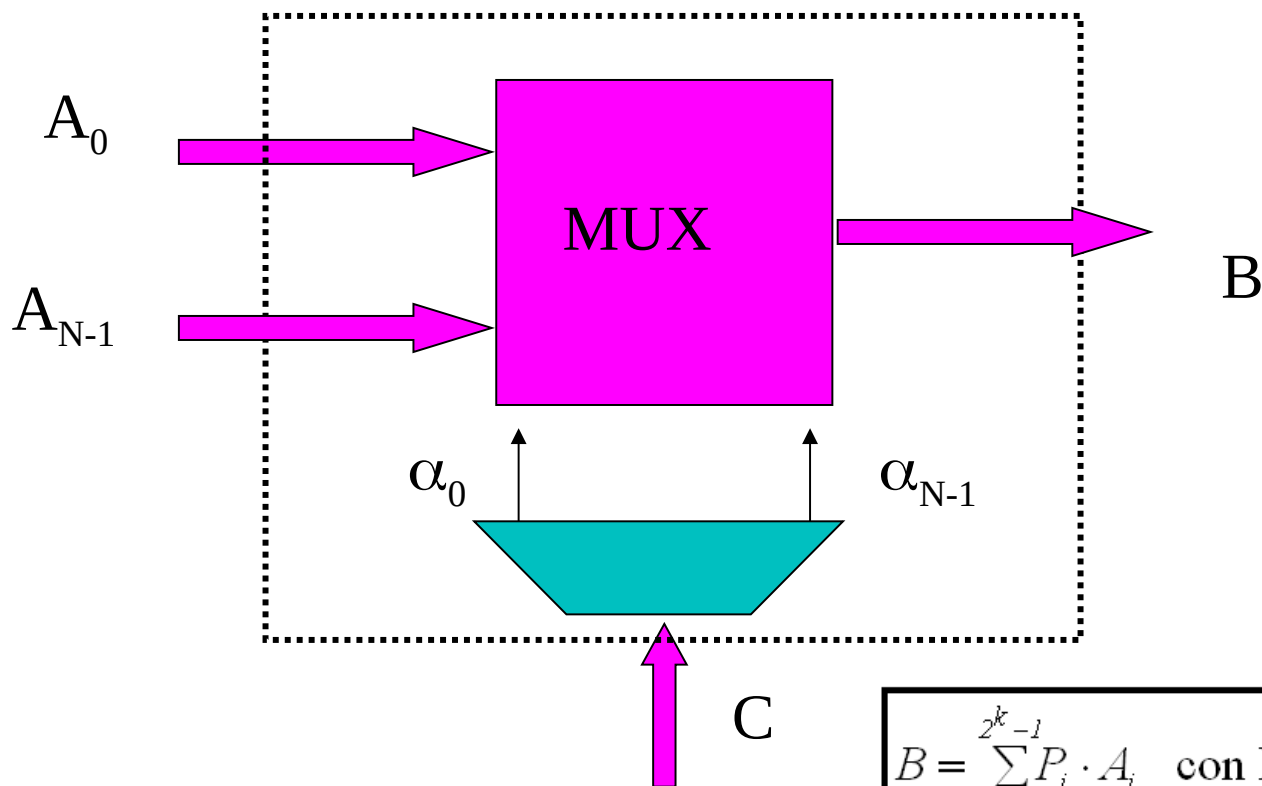


$$B = \sum_{i=0}^{n-1} \alpha_i \cdot A_i$$



Multiplexer Indirizzabile

- E' un Multiplexer Lineare i cui segnali di abilitazione sono collegati con le uscite di un decodificatore

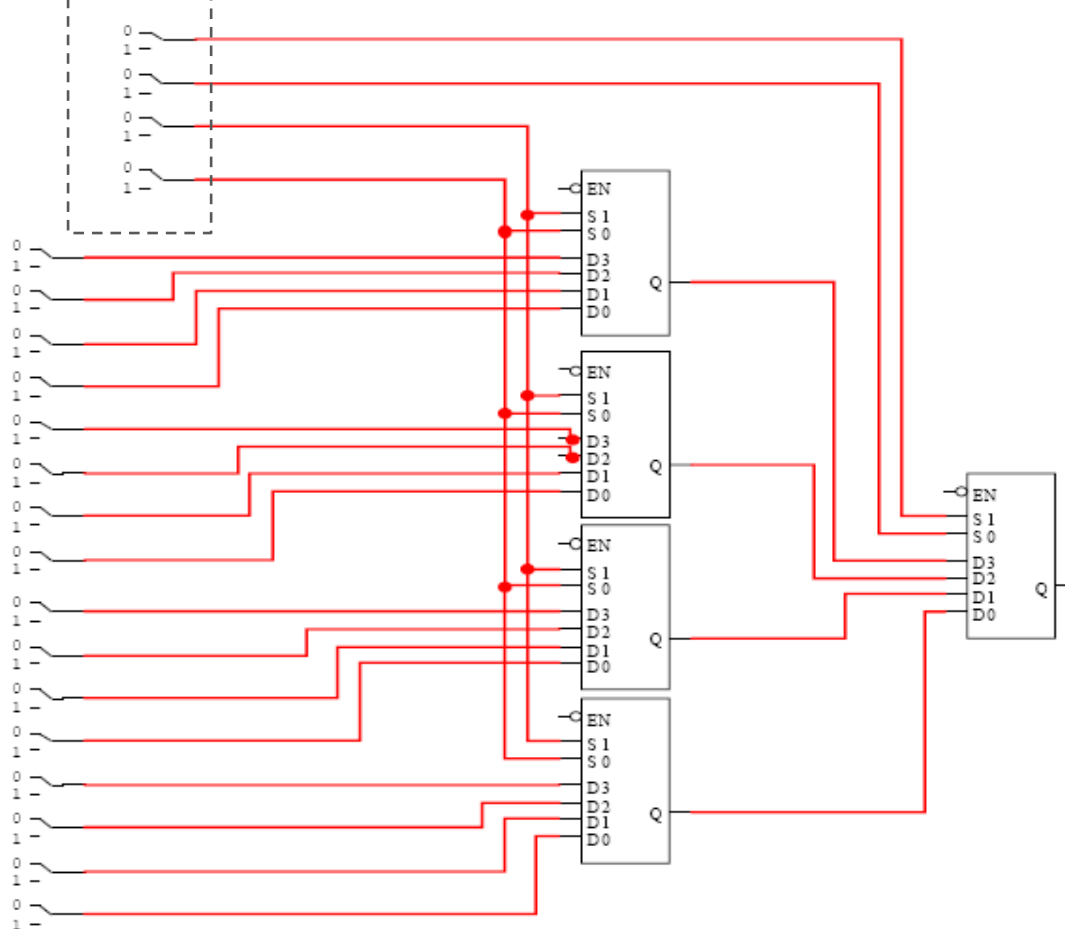


$$B = \sum_{i=0}^{2^k-1} P_i \cdot A_i \quad \text{con } P_i \text{ mintermine delle } C$$



Multiplexer composti

➤ Realizzazione di un MUX16 attraverso MUX4

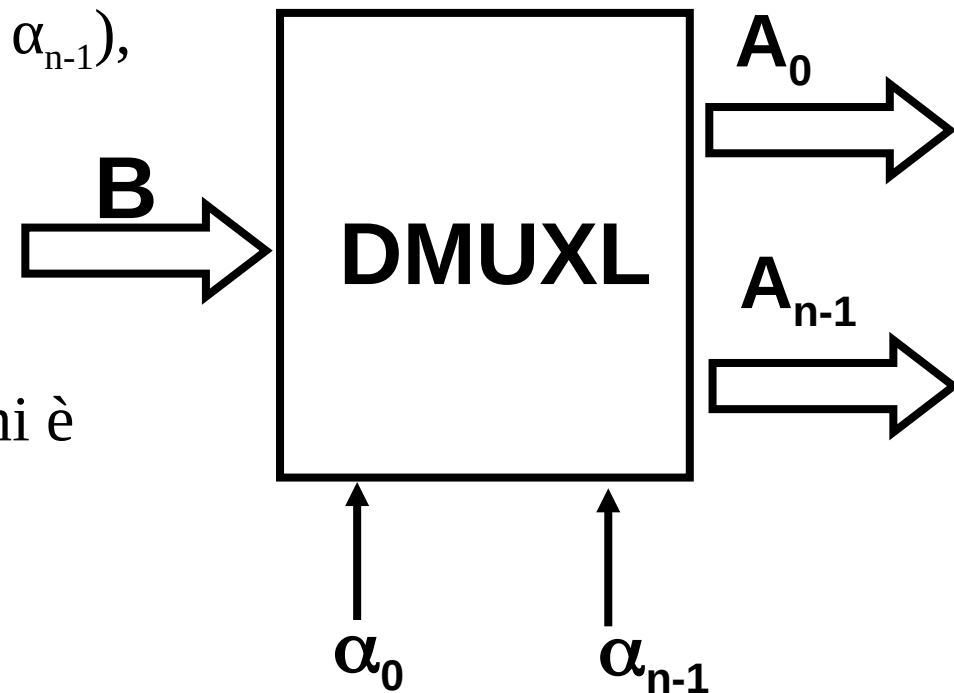


Un MUX16 può essere realizzato attraverso 5 MUX4.

Due degli ingressi di selezione consentono di scegliere il mux da leggere mentre gli altri 2 consentono di selezionare l'ingresso per ciascun mux.

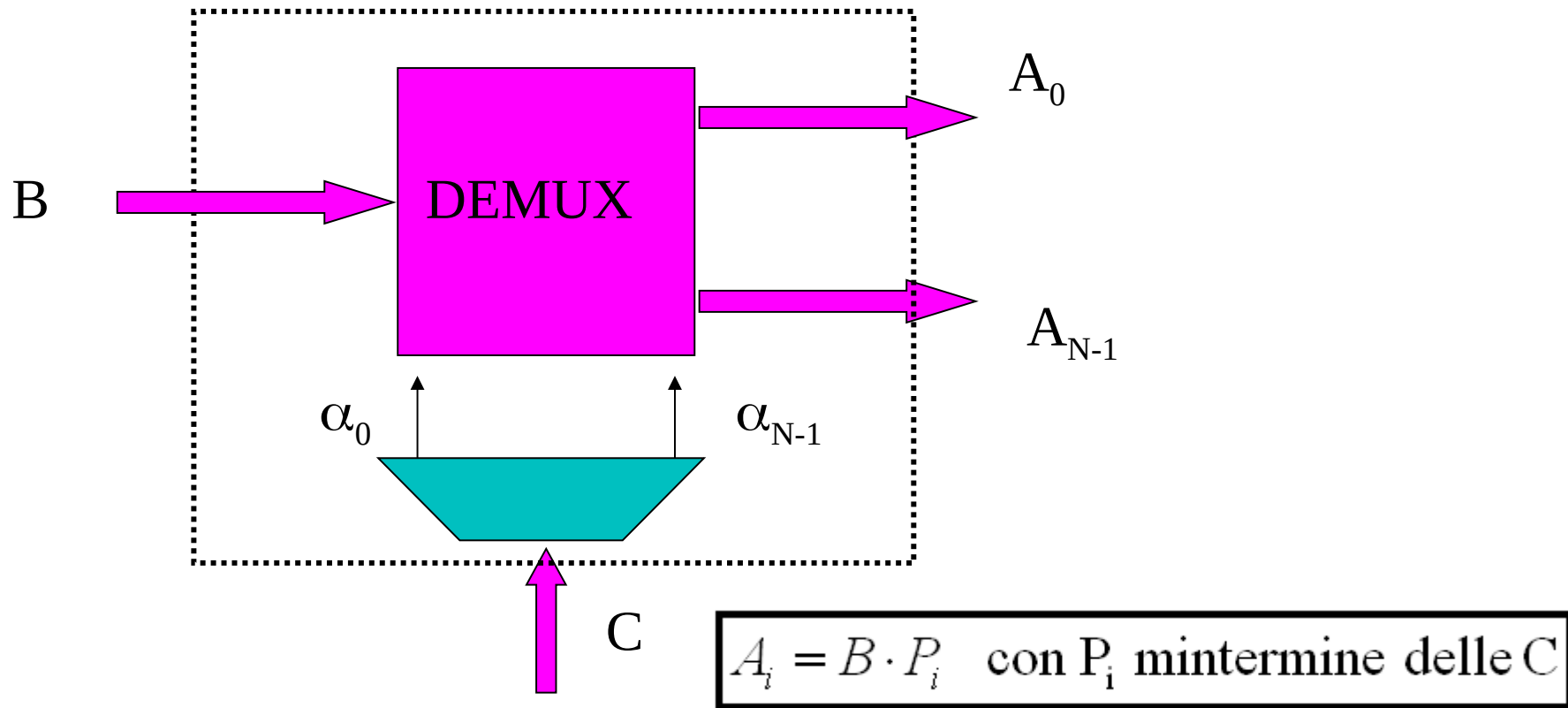
Demultiplexer lineare

- Un *Demultiplexer Lineare* (DL) è una macchina con:
 - » 1 ingresso-dati B
 - » n segnali binari di selezione ($\alpha_0, \dots, \alpha_{n-1}$), dei quali al più uno è attivo
 - » n uscite-dati ($A_0, \dots, A_{n-1}$), con
 - ◆ $A_i = B$ se è attivo α_i
 - ◆ neutro se nessuna delle selezioni è attiva



Demultiplexer Indirizzabile

- E' un Demultiplexer Lineare i cui segnali di abilitazione sono collegati con le uscite di un decodificatore



XOR ED EQ



Funzioni XOR ed EQ (1/3)

- Le funzioni XOR ($\oplus$) ed EQ ($\equiv$) assumono una notevole rilevanza dal punto di vista pratico
 - XOR: rappresenta la cifra-somma delle cifre-addendi x ed y , *somma modulo 2*.
 - Assume particolare importanza in aritmetica
 - EQ: consente di generare una variabile che indica se due bit sono uguali o diversi.

$$x \oplus y = \overline{x \equiv y} = x\bar{y} + \bar{x}y = (x + y)(\bar{x} + \bar{y})$$

$$x \equiv y = \overline{x \oplus y} = xy + \bar{x}\bar{y} = (x + \bar{y})(\bar{x} + y)$$

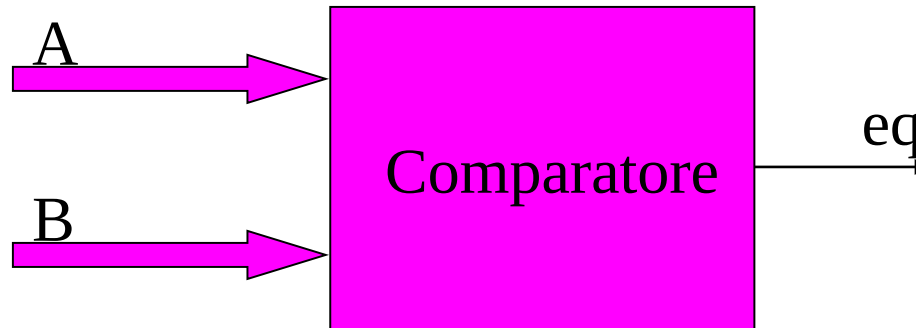


COMPARATORI



Comparatore

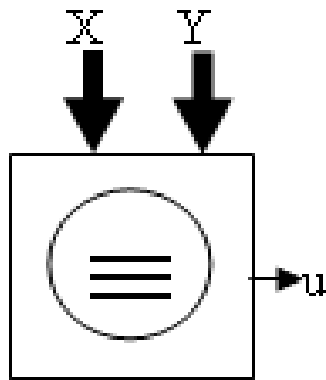
- Spesso accade che occorra confrontare due dati A e B in quanto l'unità di controllo dell'architettura deve proseguire con algoritmi differenti a seconda che i due dati siano eguali o diversi.
- *Il comparatore è una macchina che ha in ingresso due dati (A, B) ed in uscita un segnale booleano **eq** che è 1 se è $A=B$.*



Comparatori

Eguaglianza bit a bit in parallelo

- Confronto tra due stringhe di bit (parole-codice o numeri)



$$u = \prod_i (X_i \equiv Y_i)$$

Soluzione parallela:

- per n bit, n porte EQ ed una AND ad n ingressi

COMPARATORI BIT-SERIALE (per un singolo bit)

Per $A_n = B_n$ la tabella della verità è:

A_n	B_n	E_n
0	0	1
0	1	0
1	0	0
1	1	1

Con il pedice n si vuole distinguere l'n-esimo bit in una parola o numero presentato con più bit in serie nel tempo. Da cui:

Cioè E_n è alto quando A_n e B_n sono uguali.

A_n è minore di B_n se ,

A_n è maggiore di B_n se .

$$E_n = \bar{A}_n \bar{B}_n + A_n B_n = \overline{\bar{A}_n B_n + A_n \bar{B}_n}$$

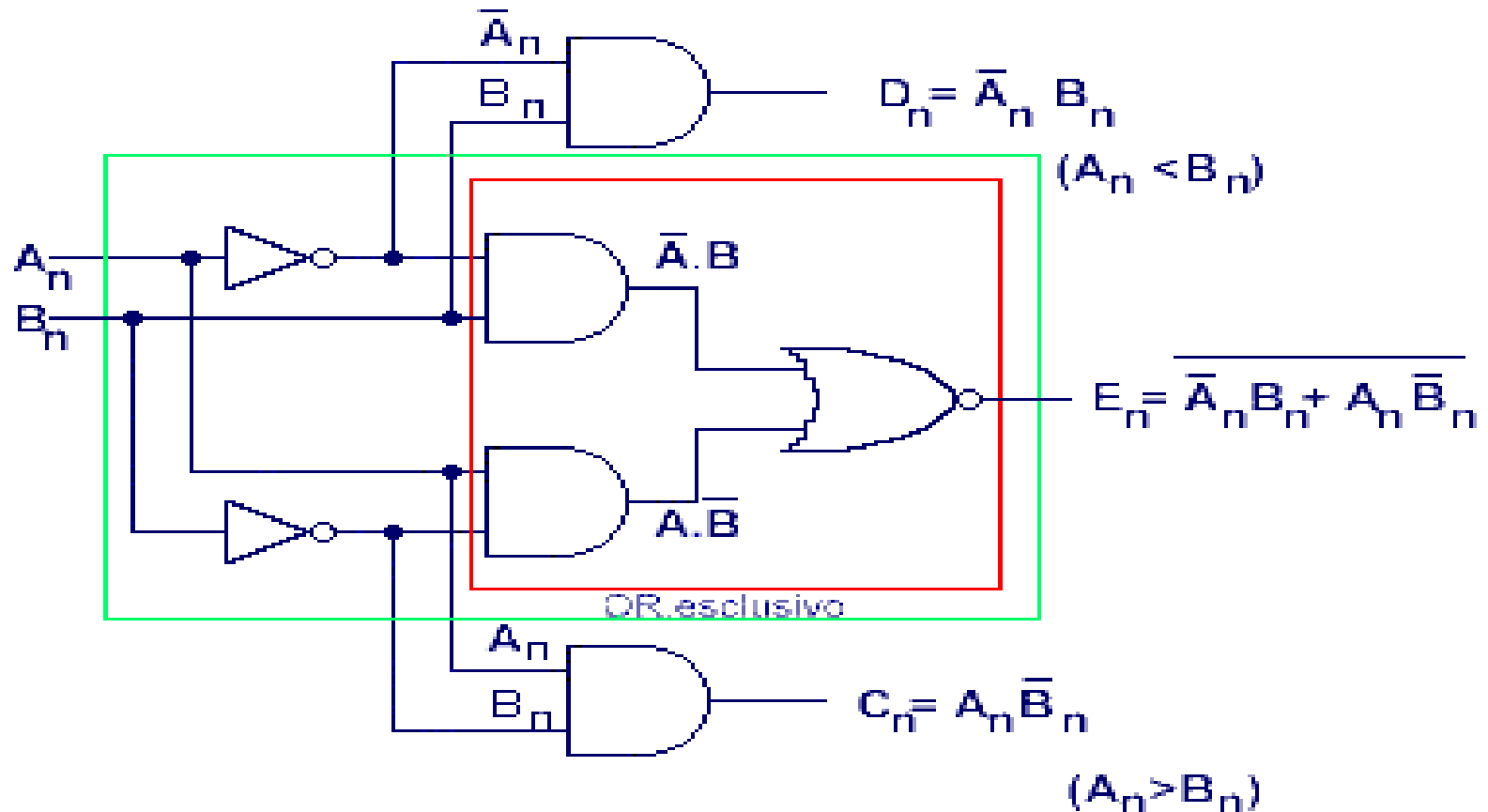
$$D_n = \bar{A}_n B_n = 1$$

$$C_n = A_n \bar{B}_n = 1$$



COMPARATORI BIT-SERIALE (per un singolo bit)

- Il circuito comparatore per il singolo bit-serie nel suo insieme ha tre uscite ed è schematizzato come in figura (NON OTTIMIZZATO):

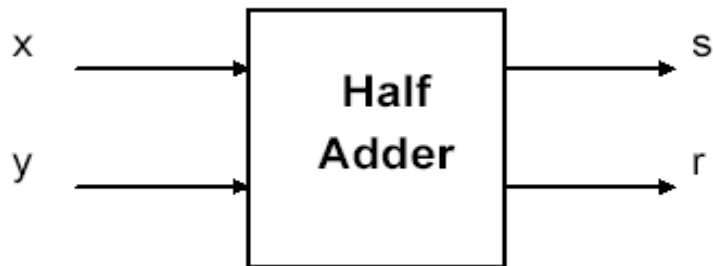


Adder



Half Adder

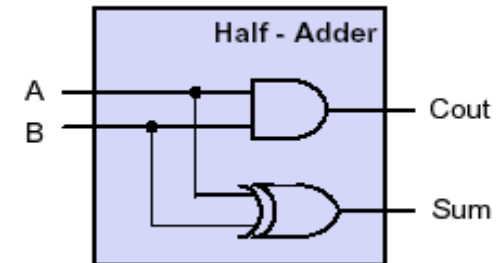
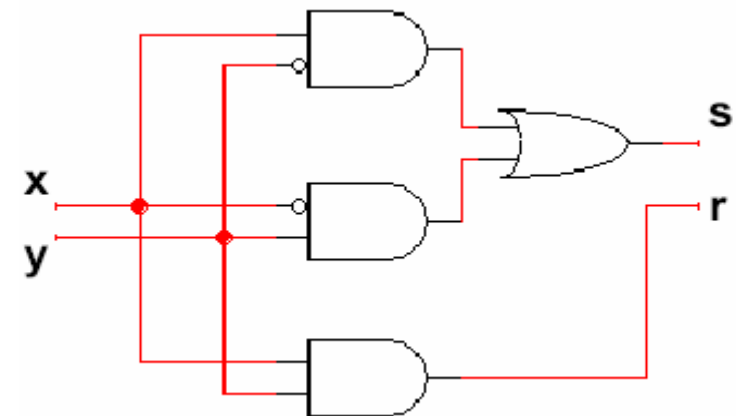
- Scrivere la tabella di verità relativa alla somma di due addendi. Riportare sia l'uscita somma che quella riporto. Disegnare anche il circuito equivalente.



x	y	s	r
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

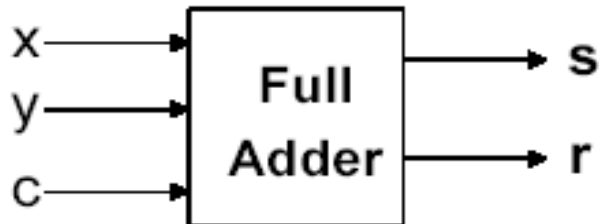
$$S = \overline{X}Y + X\overline{Y} = X \text{ xor } Y$$

$$r = XY$$



Full Adder (1/2)

- Scrivere la tabella di verità relativa alla somma di due addendi più il riporto entrante. Riportare sia l'uscita somma che quella riporto.



X	Y	r	S	R
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

xy \ r	00	01	11	10
0		1		1
1	1		1	

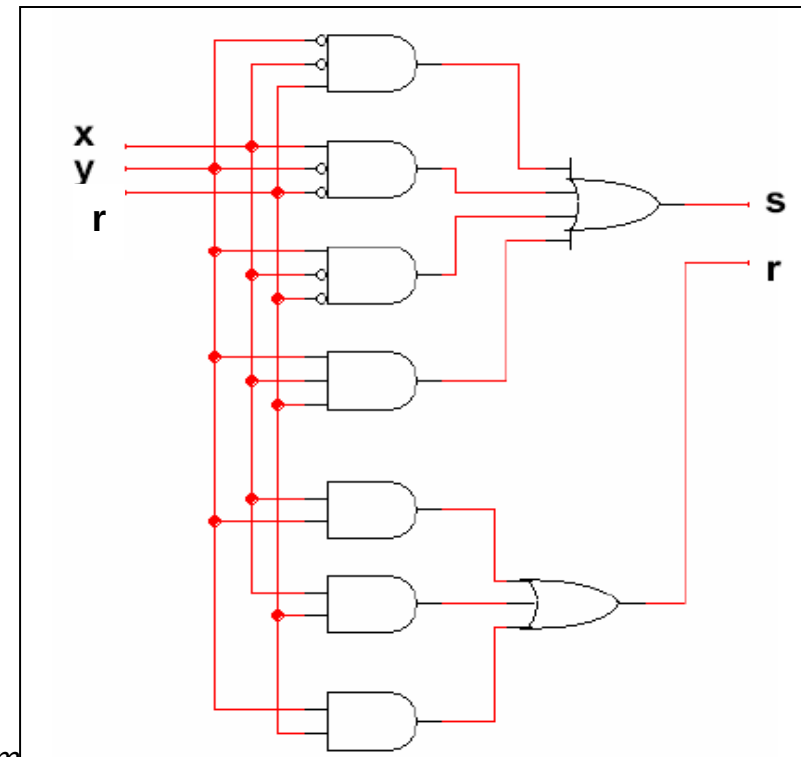
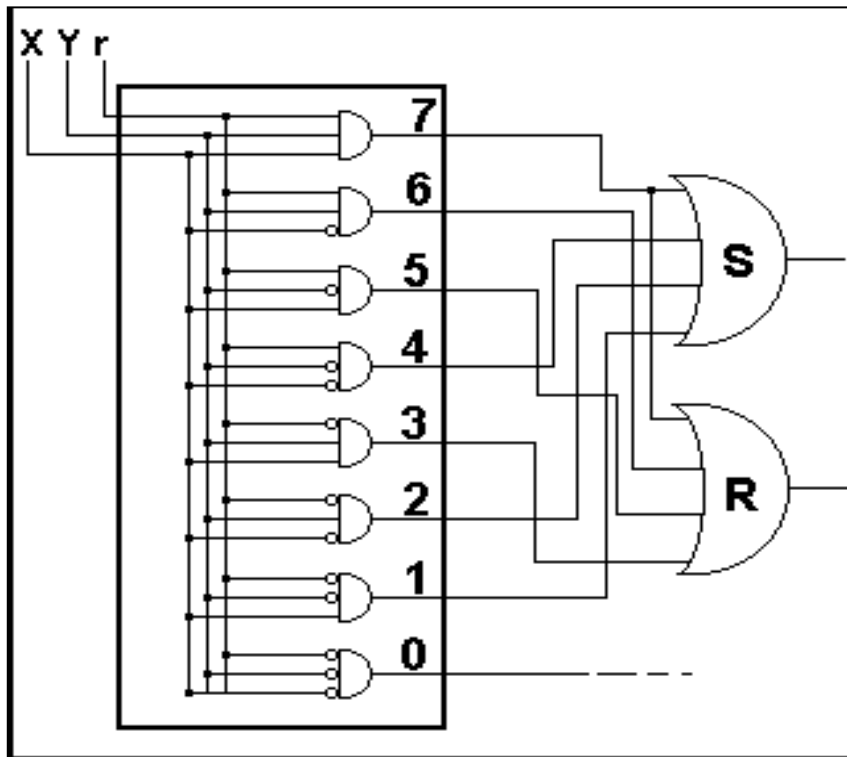
xy \ r	00	01	11	10
0			1	
1		1	1	1

Full Adder (2/2)

Le uscite valgono quindi:

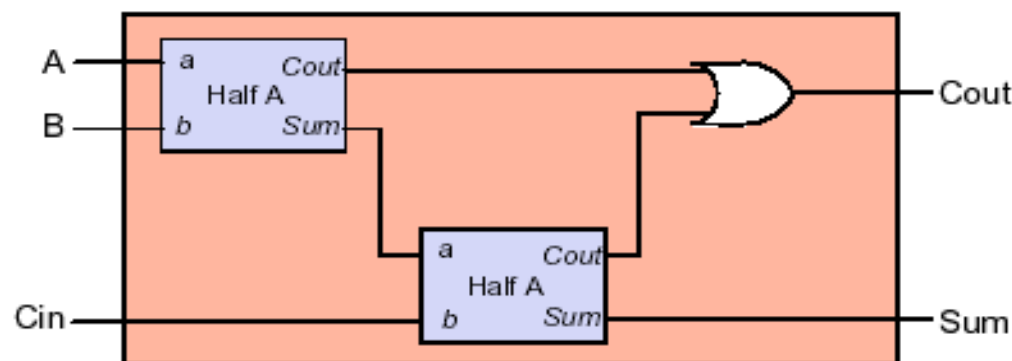
$$S = (\bar{X} \bar{Y} r) + (\bar{X} Y \bar{r}) + (X \bar{Y} \bar{r}) + (X Y r)$$

$$R = \bar{X} Y r + X \bar{Y} r + X Y \bar{r} + X Y r = X Y + Y r + X r$$



Full Adder: sintesi per decomposizione

- Gli **output** sono uguali all'Half - Adder, cioè la somma (Sum) ed il riporto (Cout). La presenza del riporto in ingresso si spiega con la possibilità di collegare N Full - Adder in serie per ottenere Full - Adder a N bit.
- La logica del dispositivo è la seguente:
 - $\text{Sum} = (A \oplus B) \oplus \text{Cin}$
 - $\text{Cout} = A * B + (A \oplus B) * \text{Cin}$e il conseguente schema logico è rappresentato in Figura.
- Si può descrivere il FA utilizzando la definizione di HA.



Addizionatore binario

- E' possibile isolare il fattore $(a \oplus b)$
- Rielaborando le precedenti espressioni è quindi possibile ottenere le seguenti espressioni per l'addizionatore completo:

$$S = (a \oplus b) \oplus r = H \oplus r$$

$$R = ab + r(a \oplus b) = G + rH$$

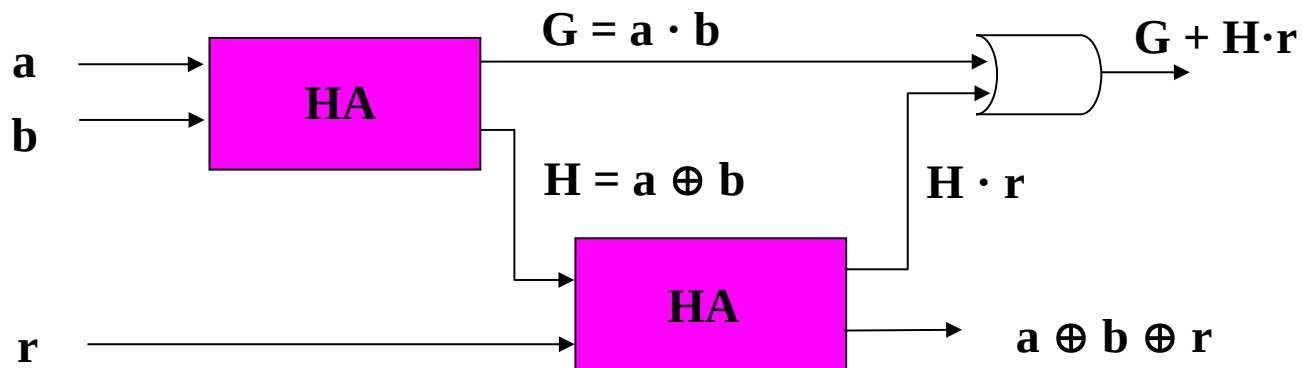


Addizionatore binario

- Pertanto, un addizionatore completo può essere ottenuto a partire da due semiaddizionatori:

$$S = (a \oplus b) \oplus r = H \oplus r$$

$$R = ab + r(a \oplus b) = G + rH$$



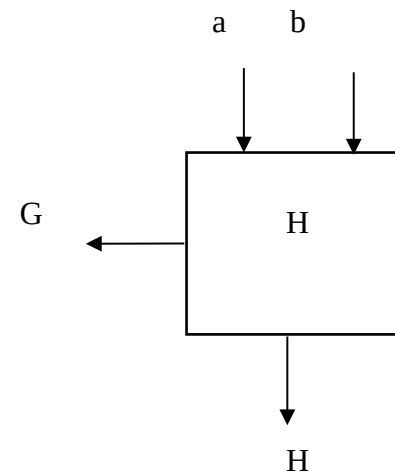
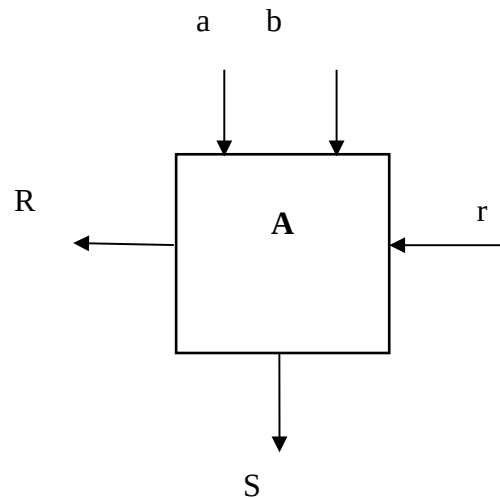
Addizionatore binario: il riporto

- Le diverse componenti dell'espressione ottenuta per l'addizionatore binario assumono un significato particolare:
 - » **$G = a \cdot b$ “riporto generato”**: indica la creazione di un riporto all'interno dell'addizionatore binario
 - » **$P = H = a \oplus b$ “riporto propagato”**: indica se, in presenza di un riporto in ingresso, lo stesso verrà propagato in uscita
 - » Il riporto in uscita può quindi essere espresso come **$R = G + P_r$**



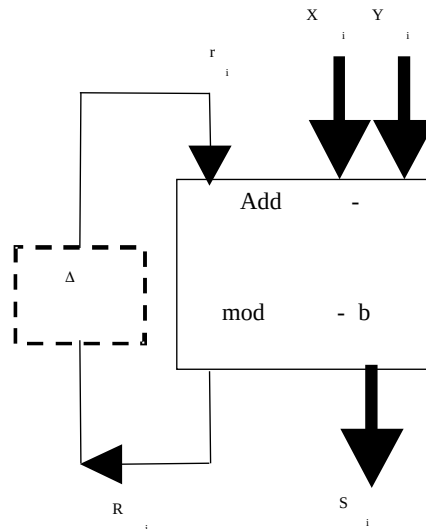
Addizionatore binario: conclusione

- Addizionatore completo (full-adder)
 - » ha il riporto entrante
- Semi-addizionatore (half-adder)
 - » non include il riporto entrante



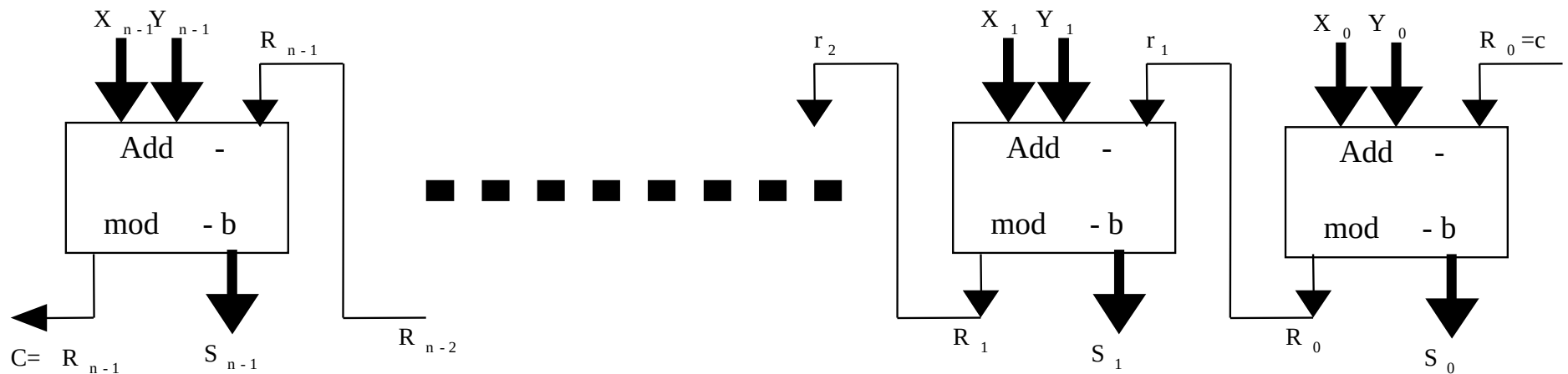
Addizionatori seriali

- Usa un unico addizionatore operante sulla singola cifra
- Opera in momenti successivi su cifre diverse degli addendi
- Richiede un blocco “con memoria”
- E' lento rispetto ad addizionatori che lavorano in parallelo sulle diverse cifre degli addendi



Addizionatori paralleli

- Opera sulle cifre degli addendi in parallelo
- ...anche se il riporto deve propagarsi attraverso l'intera struttura
- Richiede un numero maggiore di risorse rispetto all'addizionatore seriale



Addizionatori paralleli

- Gli addizionatori ottenuti collegando in cascata n addizionatori di cifra sono anche chiamati addizionatori a propagazione del riporto (*carry-ripple* o *carry-propagate*)
- ε = tempo di risposta di uno stadio
- Allo stadio i , il riporto uscente o è **generato** o è **ucciso** o è **propagato**
- Tempo di ritardo complessivo: **Limite inferiore** ε (in tutti gli stadi il riporto è generato o ucciso)
- Tempo di ritardo complessivo: **Limite superiore** $n\varepsilon$ (un riporto entrante nel primo stadio che è propagato in tutti gli stadi)
- Tempo di ritardo complessivo = $k\varepsilon$ ($k \leq n$), dove k è la più lunga catena di condizioni di propagazione.



Addizionatori con anticipo del riporto

- Normalmente chiamati addizionatori *carry lookahead*
- Per ogni stadio i , dal $(k+1)$ -esimo al $(k+j)$ -esimo, r_i si ottiene direttamente dai bit degli addendi X ed Y e dal riporto entrante nella catena (invece che dal riporto uscente R_{i-1})

$$r_{k+1} = G_k + r_k P_k$$

$$r_{k+2} = G_{k+1} + G_k P_{k+1} + r_k P_k P_{k+1}$$

.....

$$r_{k+j} = G_{k+j-1} + G_{k+j-2} P_{k+j-1} + \dots + G_k P_{k+1} \dots P_{k+j-1} + r_k P_k P_{k+1} \dots P_{k+j-1}$$



Addizionatori carry lookahead

$$r_{k+j} = G_{k+j-1} + G_{k+j-2}P_{k+j-1} + \dots + G_k P_{k+1} \dots P_{k+j-1} + r_k P_k P_{k+1} \dots P_{k+j-1}$$

r_{k+j} è alto se è verificata la condizione di generazione nell'ultimo stadio

....
....

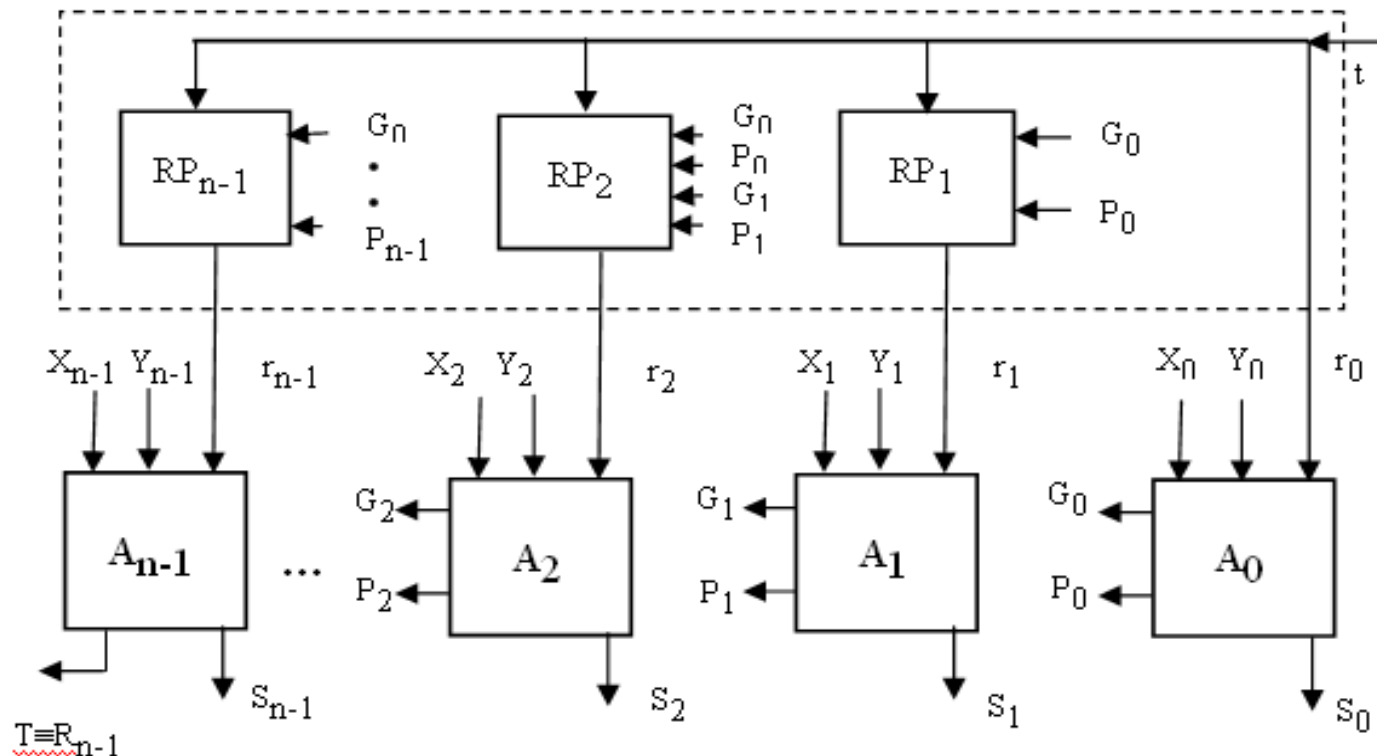
...oppure se è verificata la condizione di generazione G_k e se questa viene propagata dagli stadi dal $(k+1)$ -esimo fino all'ultimo

...oppure è pari al riporto entrante r_k se questo viene propagato in tutti gli stadi



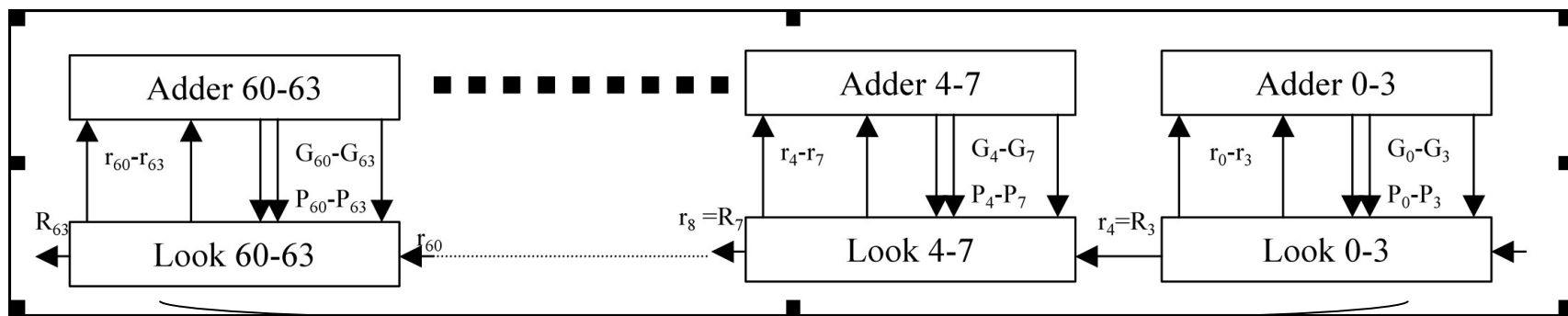
Addizionatori carry lookahead

- L'espressione precedente può essere realizzata nella maniera riportata in figura



Addizionatori carry lookahead

- L'idea di base negli addizionatori carry lookahead è quella di calcolare la relazione tra r_k ed r_{k+j} separatamente per ogni gruppo di cifre $k+1 \dots k+j$
- Una rete a livello superiore valuta la propagazione del carry tra *gruppi* di cifre
- Ad esempio, per un adder a 64 bit suddiviso in blocchi di quattro cifre (bit), la parte di “look” lungo cui si propaga il riporto è lunga $64/4=16$ stadi



16 blocchi



Addizionatori carry lookahead

